

Netflix

Spotify

Subscription

YouTube

Basic-Eit

it gives us a sense of
independence...

Feeling Young...

- Forgetting
- Instantly sense
- Something very basic

- Forgetting
- Instantly sense
- Something very basic

Skin

Tight

Smooth

Fresh

F O R

≠

E V E R

Wrinkles

Sagging

Spots



AGE



Health
&
energy



First & last
layer
outfit



How to protect?

Skin types

Normal Dry Oily Diseased

MARKET

Dior
Ordinary
HMN Clinique Nivea
Fresh

Common problems

What should I buy?

Solution

With just a photo, the app scans your skin and tells you your skin type. Based on that, it recommends the right kind of products specifically for you

Preprocessing

VGG16

• JPG

• Blurry

• Duplicate

```
import cv2
import os
import numpy as np

# Function to check if an image is blurry
def is_blurry(image_path, threshold=100):
    image = cv2.imread(image_path)
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) # Convert to grayscale
    laplacian = cv2.Laplacian(gray, cv2.CV_64F) # Apply Laplacian operator
    variance = laplacian.var() # Compute variance

    return variance, variance < threshold # Return variance and blur status

# Path to dataset folder
dataset_path = r"C:\Users\Asus\Desktop\final data set\test dataset"

# Check all images
blurry_images = []
sharp_images = []
image_variances = []

for class_folder in os.listdir(dataset_path):
    class_path = os.path.join(dataset_path, class_folder)

    if os.path.isdir(class_path): # Ensure it's a directory
        for image_name in os.listdir(class_path):
            image_path = os.path.join(class_path, image_name)
            variance, is_blur = is_blurry(image_path, threshold=100)

            image_variances.append(variance)

            if is_blur:
                blurry_images.append((image_path, variance))
            else:
                sharp_images.append((image_path, variance))

# Display some results
print(f"♦ Total Images Checked: {len(image_variances)}")
print(f"♥ Blurry Images: {len(blurry_images)}")
print(f"✅ Sharp Images: {len(sharp_images)}")
```

```
♦ Total Images Checked: 2186
♥ Blurry Images: 108
✅ Sharp Images: 2078
```

```
import os
import cv2

def convert_images_to_format(input_dir, output_dir, target_format="jpg"):
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    for class_name in os.listdir(input_dir): # Loop through class folders
        class_path = os.path.join(input_dir, class_name)
        output_class_path = os.path.join(output_dir, class_name)

        if os.path.isdir(class_path):
            if not os.path.exists(output_class_path):
                os.makedirs(output_class_path)

            for img_name in os.listdir(class_path):
                img_path = os.path.join(class_path, img_name)
                img = cv2.imread(img_path)

                if img is not None:
                    new_img_name = os.path.splitext(img_name)[0] + f".{target_format}"
                    new_img_path = os.path.join(output_class_path, new_img_name)
                    cv2.imwrite(new_img_path, img) # Save image in new format
```

```
import os
import cv2
import hashlib

def get_image_hash(image_path):
    """Generate a unique hash for an image."""
    with open(image_path, "rb") as f:
        return hashlib.md5(f.read()).hexdigest() # Compute MD5 hash

def remove_duplicates(folder_path):
    """Remove duplicate images in a given folder."""
    seen_hashes = {}
    deleted_count = 0

    for class_name in os.listdir(folder_path):
        class_dir = os.path.join(folder_path, class_name)
        if not os.path.isdir(class_dir):
            continue # Skip if not a folder

        for img_name in os.listdir(class_dir):
            img_path = os.path.join(class_dir, img_name)

            if not os.path.isfile(img_path):
                continue # Skip non-file items

            img_hash = get_image_hash(img_path)

            if img_hash in seen_hashes:
                os.remove(img_path) # Delete duplicate
                deleted_count += 1
                print(f"Deleted: {img_path}")
            else:
                seen_hashes[img_hash] = img_path # Store unique image

    print(f"✅ Finished! Deleted {deleted_count} duplicate images.")

# Set the dataset path
dataset_path = r"C:\Users\Asus\Desktop\final data set\Sharp"

# Run the function
remove_duplicates(dataset_path)
```

```
✅ Finished! Deleted 0 duplicate images.
```



```

# Add your code to preprocess the dataset for the CNN model.
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Define paths
normal_dataset_path = r"C:\Users\Asus\Desktop\final data set\split\train"
val_path=r"C:\Users\Asus\Desktop\final data set\split\val"
test_path=r"C:\Users\Asus\Desktop\final data set\split\test"

# Define ImageDataGenerators
train_datagen = ImageDataGenerator(
    rescale=1./255,
    # rotation_range=20, # Rotate images randomly # Shift vertically
    # shear_range=0.2, # Apply shearing
    # zoom_range=0.2, # Random zoom
    # horizontal_flip=True, # Flip horizontally
    # fill_mode='nearest' # Fill missing pixels # Normalize pixel values (0-1)
)

# Load Augmented Dataset
train_generator = train_datagen.flow_from_directory(
    normal_dataset_path,
    target_size=(224, 224), #it is predefined for this model
    batch_size=16,
    class_mode='categorical' # Use 80% of data for training
)

val_generator = train_datagen.flow_from_directory(
    val_path,
    target_size=(224, 224),
    batch_size=16,
    class_mode='categorical',
    shuffle=False # Use 20% for validation
)

test_generator = train_datagen.flow_from_directory(
    test_path,
    target_size=(224, 224),
    batch_size=16,
    class_mode='categorical',
    shuffle=False # Do not shuffle test data for consistent evaluation
)

```

```

# Add your code to create a CNN architecture for your image dataset.

from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Flatten, Dense, Dropout, BatchNormalization

# Load VGG16 model without top layers
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Freeze all convolutional layers (Feature Extraction)
for layer in base_model.layers:
    layer.trainable = False

# Add Custom Fully Connected Layers for Skin Type Classification
x = Flatten()(base_model.output)
x = Dense(512, activation='relu')(x)
x = BatchNormalization()(x)
x = Dropout(0.6)(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)
output_layer = Dense(4, activation='softmax')(x) # 4 classes

# Create Model
model = Model(inputs=base_model.input, outputs=output_layer)

```

I used a learning rate of 0.00003, set accuracy as a metric, and used categorical_crossentropy as the loss function since I had four classes. I also implemented early stopping to stop training if the model started performing well within the first few epochs.

```
# Add your code to train the CNN model using proper loss function
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.callbacks import ReduceLROnPlateau

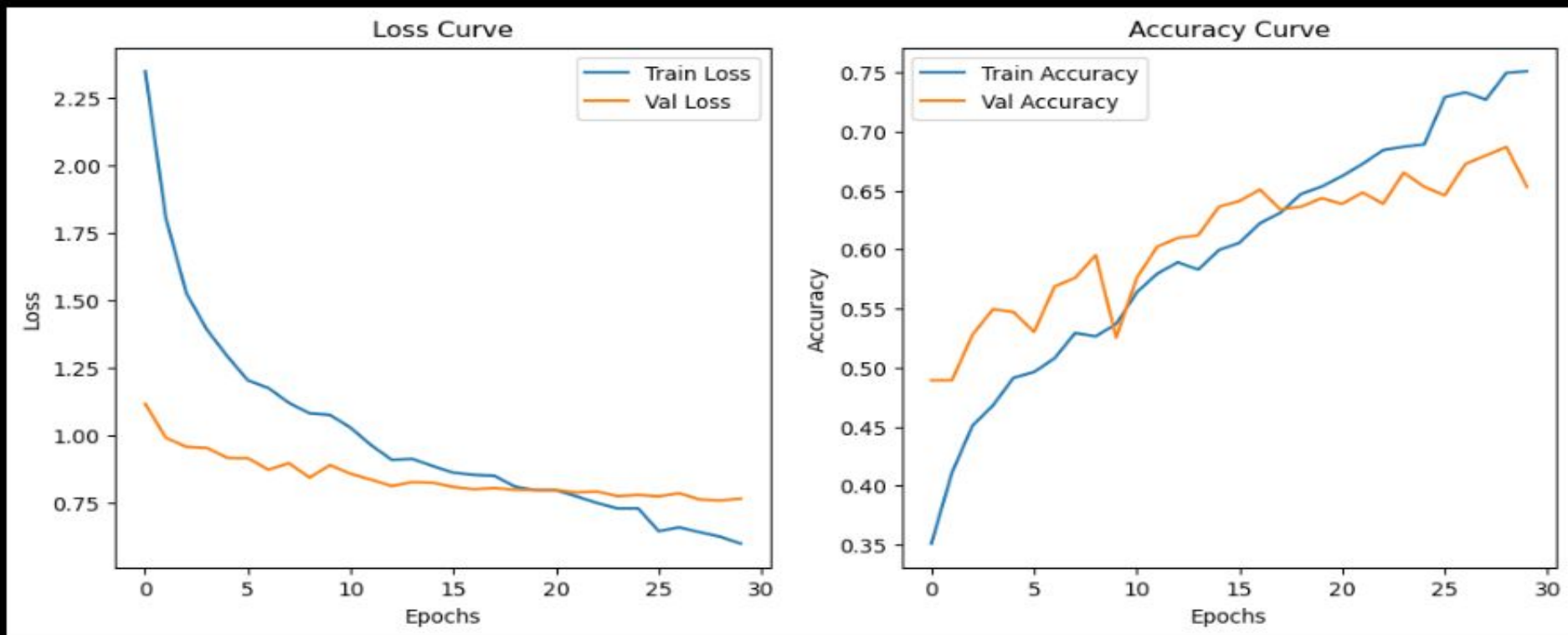
for layer in base_model.layers[-6:]: #-12
    layer.trainable = True
# Compile Model
model.compile(
    optimizer=Adam(learning_rate=0.00003),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Early Stopping
early_stopping = EarlyStopping(
    monitor='val_loss', patience=3, restore_best_weights=True
)

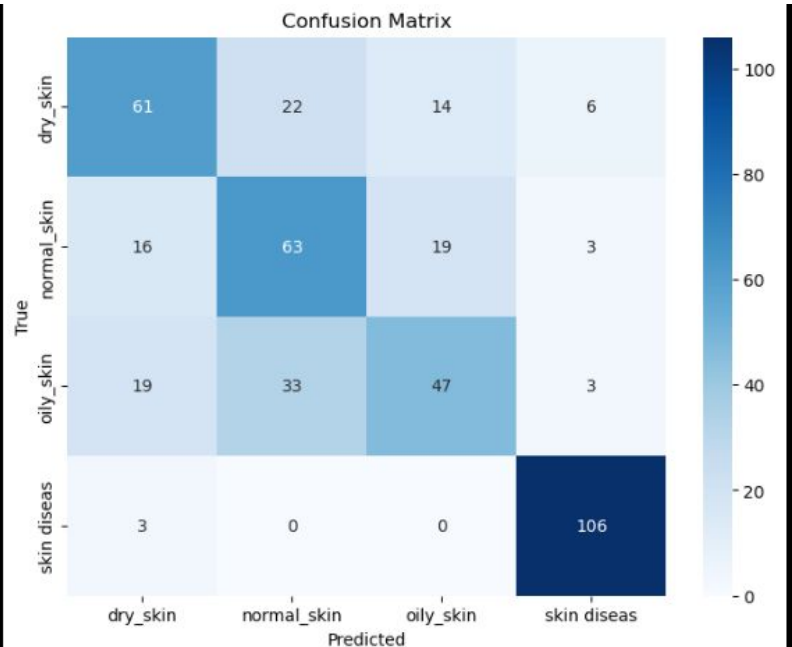
lr_scheduler = ReduceLROnPlateau(
    monitor='val_loss', patience=2, factor=0.5, verbose=1
)
```

14/14 [=====] - 8s 552ms/step - loss: 0.7666 - accuracy: 0.6476
26/26 [=====] - 16s 584ms/step - loss: 0.7673 - accuracy: 0.6530

- ✅ Test Loss: 0.7666
- ✅ Test Accuracy: 56.19%
- 🔥 Best Model Performance at Epoch 29
- ✅ Best Validation Accuracy: 68.67%
- ✅ Best Validation Loss: 0.7602



Confusion Matrix



Classification Report:

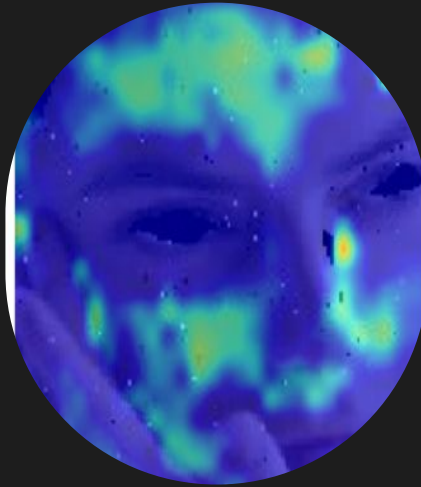
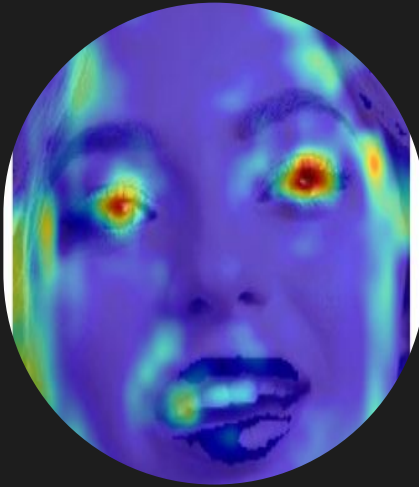
	precision	recall	f1-score	support
dry_skin	0.62	0.59	0.60	103
normal_skin	0.53	0.62	0.58	101
oily_skin	0.59	0.46	0.52	102
skin diseases	0.90	0.97	0.93	109
accuracy			0.67	415
macro avg	0.66	0.66	0.66	415
weighted avg	0.66	0.67	0.66	415

Explainable AI

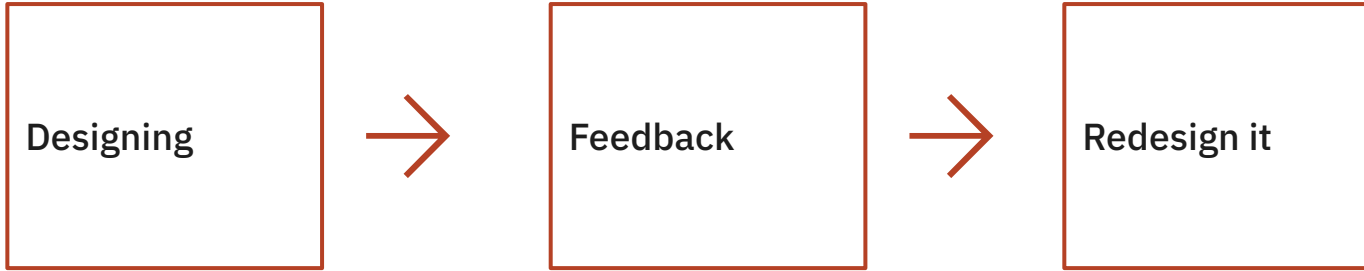
Open the Black Box

Grad-CAM

What is happening inside



Think Aloud Study



2. What features or changes would you suggest improving the system?

I think it can be more straightforward like the back buttons and here the buttons can be in a good size. Like the text is sometimes too small that you can see.

I think just aesthetics, maybe. Or even when you go back on this button, does this take a picture? No, just cut a picture from the gallery. Oh, okay. So this could have an option as upload your skin image from your gallery or take a picture. All right. Just change it. Exactly. So you know it's more efficient and people can understand it better. Right. I think that's it.

Maybe you can a little bit put more big sign in here. Oh, okay. For the menu part. Yeah, for the menu part. But the other part, I think the overall looks good. Great.

Final Demo

